



# PRD: Rival Restaurant — Competitive Service Management Game

**Rival Restaurant** is a real-time, head-to-head restaurant-management game where two players compete for a shared pool of customers in a dynamic neighborhood. The strategic layer happens before each round—choosing menu, prices, inventory, and staffing priorities—while the active layer puts each owner on the restaurant floor, cooking, restocking, delivering orders, resolving bottlenecks, and purchasing upgrades.

The game should feel like a competitive operations game, not a clicker: players win by reading the local market, committing to a viable service strategy, and personally intervening where their restaurant is under pressure.

## 1. Product overview

### Working title

**Rival Restaurant**

Alternative names:

- Service Wars
- Dinner Rush
- Table Stakes
- Two Kitchens
- Market & Maitre D'
- The Dinner District

### Genre

- Real-time competitive restaurant-management game
- 1v1 multiplayer operations/strategy game
- Third-person or high-angle isometric 3D simulation
- Short-session competitive game, initially optimized for browser play

## Target platform

- Desktop web first
- Mouse and keyboard controls
- Mobile/tablet is out of scope for the initial release, though UI should not make later adaptation impossible

## Technical constraints

- Client: JavaScript/TypeScript, React for UI, Three.js for 3D
- Three.js must load from a pinned CDN import, not be bundled into the application
- Server: plain JavaScript Express application
- Real-time multiplayer: native WebSockets, preferably using the lightweight ws library
- No Next.js
- No monorepo framework
- No game engine such as Unity, Unreal, Phaser, Babylon, Godot, or PlayCanvas
- Client and server must live in separate folders in a conventional repository
- 3D scene harnesses must be first-class development artifacts so visual systems can be edited and reviewed without launching a full multiplayer match
- External/open 3D assets are acceptable where their license permits reuse and attribution requirements are met

A CDN/import-map approach is appropriate for browser-side Three.js module loading; bare imports such as `import * as THREE from 'three'` need an import map in a browser environment, and all Three.js modules should use the same pinned version/CDN to avoid incompatibilities. Express can host the HTTP API and static client files while ws handles low-overhead WebSocket game connections. <sup>[1]</sup> <sup>[2]</sup> <sup>[3]</sup> <sup>[4]</sup>

## 2. Product vision

Create a restaurant game where the question is not merely, “Can I serve food quickly?” but:

“Given this neighborhood, this time slot, these nearby events, this customer mix, my rival’s strategy, and my physical restaurant state—what should I sell, at what price, and where should I personally intervene right now?”

The game combines three forms of competitive mastery:

### 1. Market reading

Understand who is likely to arrive, what they want, what price they will tolerate, and how the district conditions are changing.

### 2. Restaurant configuration

Build a menu, set prices, provision ingredients, and select strategic upgrades before service begins.

### 3. Floor execution

Move through the restaurant in real time to help workers, eliminate service bottlenecks, deliver food, replenish supplies, handle dissatisfied guests, and buy upgrades at the right moment.

The player should feel like an owner-operator: one part restaurateur, one part line cook, one part expeditor, and one part competitive strategist.

### 3. Core player fantasy

“I out-read my rival, built the better menu for this exact neighborhood and event, then personally kept my operation from falling apart during the rush.”

The moment-to-moment fantasy should include:

- Seeing a wave of concert attendees or baseball fans enter the district.
- Realizing the player’s menu is suddenly positioned well—or badly.
- Sprinting from the dining room to the kitchen to help finish a high-value rush order.
- Choosing between personally delivering a premium dish, restocking a critical ingredient, or buying an upgrade while cash is available.
- Watching a rival win customers through a lower price, faster service, stronger reputation, or a better market fit.
- Recovering from a bad setup through strong execution rather than losing automatically.

## 4. Design pillars

### 4.1 Active ownership, not passive clicking

The player has direct embodied control over the restaurant owner. The owner can move, carry objects, cook, deliver, clean, restock, resolve incidents, and purchase upgrades.

There should be no primary “click repeatedly to create money” loop. Every meaningful action should map to an in-world operational decision.

### 4.2 Competition through a shared market

Both restaurants draw from the same customer population. One restaurant succeeding should often affect the other:

- Faster service can capture undecided customers.
- Better pricing can attract price-sensitive segments.
- A stronger menu match can attract preference-driven customers.
- Capacity failures can send customers to the competitor.
- An event can favor one menu, cuisine, price level, or service style.

- Reputation can compound across a match, but not so strongly that the match becomes unwinnable early.

### **4.3 Strategic setup, chaotic execution**

The setup phase rewards forecasting and planning. The service phase tests whether that plan can survive reality.

Players should not be able to fully optimize both phases independently. A great menu can still fail due to poor floor management; excellent execution can partially overcome a mediocre menu.

### **4.4 Readable 3D simulation**

The 3D world should communicate operational state at a glance:

- Hungry/waiting customers visibly look impatient.
- Tables show order, meal, payment, and cleanup states.
- Stations show backlog and ingredient shortages.
- Workers display current jobs.
- Bottlenecks use clear visual language: queue icons, heat/urgency colors, progress bars, and customer body language.
- The competitor's restaurant should be visible in some form, either physically adjacent, across a street, or via a district overview panel.

### **4.5 Short, replayable matches**

Initial target: 8–12 minute matches.

A round should support a complete strategic arc:

1. Learn the market.
2. Make pre-service decisions.
3. React to events.
4. Build or lose momentum.
5. Make late-match tactical upgrades.
6. See a clear winner.

## **5. Game loop**

## Match structure

| Phase         | Duration        | Player activity                                                              | Purpose                        |
|---------------|-----------------|------------------------------------------------------------------------------|--------------------------------|
| Lobby         | Variable        | Join/create room, select cosmetic identity, ready up                         | Establish 1v1 match            |
| Market Reveal | 20–30 sec       | Read district, customer segments, nearby businesses, initial forecast        | Form initial strategy          |
| Setup Phase   | 90–120 sec      | Build menu, set prices, allocate starting inventory, choose initial upgrades | Commit to a business plan      |
| Service Phase | 6–8 min         | Run restaurant, move owner, help staff, respond to events, buy upgrades      | Core active competition        |
| Final Rush    | Final 60–90 sec | Handle intensified traffic/event conditions and finalize score               | Create climax                  |
| Results       | 30–60 sec       | Compare score, revenue, satisfaction, customers, tactics                     | Reinforce learning and rematch |

## Match pacing target

For a 10-minute total match:

- Market reveal: 30 seconds
- Setup: 2 minutes
- Main service: 6 minutes
- Final rush: 1 minute
- Results: 30 seconds

The first playable prototype may reduce this to:

- Setup: 45 seconds
- Service: 3–4 minutes
- Results: 20 seconds

This allows fast balancing and multiplayer testing before more content is added.

## Moment-to-moment service loop

During service, the player repeatedly makes choices like:

1. Observe the state of the restaurant and market.
2. Identify the most valuable current bottleneck.
3. Move the owner to that location.
4. Perform a contextual action:
  - Cook/prep a dish
  - Plate an order

- Carry food to a table
- Restock a station
- Seat a waiting party
- Clean a table
- Calm or compensate an unhappy customer
- Repair/clear a station problem
- Buy an upgrade

5. Reassess based on queues, revenue, satisfaction, the rival's visible performance, and newly announced events.

The player must regularly trade immediate operational help against strategic spending. For example, stepping away from the kitchen to buy a faster oven may be correct long-term, but it may cause three dishes currently in the queue to miss their freshness window.

## 6. Market system

### Overview

Every match is generated from a **Market Scenario**. The scenario determines the customer pool, customer behavior, initial demand, likely events, and pricing tolerance.

The initial version uses 3–5 distinct neighborhood archetypes. Each archetype is defined by:

- District identity
- Daypart
- Nearby business/event anchors
- Customer-segment weights
- Price sensitivity
- Preferred food categories
- Expected party sizes
- Service-speed expectations
- Likely random events
- Map/restaurant visual styling

The system should be data-driven so designers can create or tune scenarios in JSON without modifying gameplay code.

## Initial neighborhood archetypes

| Market               | District / daypart    | Nearby anchors                         | Typical customers                               | Core opportunity                            | Core risk                                                  |
|----------------------|-----------------------|----------------------------------------|-------------------------------------------------|---------------------------------------------|------------------------------------------------------------|
| Uptown Pre-Theater   | Uptown, dinner        | Opera, theater, galleries              | Affluent couples, visitors, professionals       | Premium dishes and high-margin add-ons      | Customers have low tolerance for late service before shows |
| Downtown Lunch Rush  | Downtown, lunch       | Offices, transit, retail               | Office workers, delivery-minded diners, groups  | Fast, affordable, consistent meals          | High queue pressure and price sensitivity                  |
| Stadium District     | Evening, event day    | Baseball stadium, sports bars, parking | Fans, families, groups, tourists                | High-volume comfort food and beverages      | Sharp pre/post-event demand spikes and messy tables        |
| Arts & Nightlife     | Evening/night         | Music venue, clubs, bars               | Young professionals, nightlife groups, tourists | Trendy dishes, snacks, drinks, late service | Demand can swing hard based on concert/event outcomes      |
| Neighborhood Weekend | Weekend brunch/dinner | Parks, apartments, local shopping      | Families, regulars, couples                     | Broad satisfaction and repeat demand        | Diverse needs make specialization less reliable            |

## Recommended MVP markets

Start with three:

1. Downtown Lunch Rush
2. Uptown Pre-Theater
3. Stadium District

These provide strong, legible contrasts:

- Speed vs. margin
- Predictable rush vs. volatile rush
- Price-sensitive diners vs. premium diners
- Solo/business guests vs. large parties
- Short dining windows vs. larger average checks

Add Arts & Nightlife and Neighborhood Weekend after the core loop is stable.

## Customer segments

The game begins with five reusable customer archetypes. Each market changes their likelihood and parameters.

| Segment              | Primary priority               | Price sensitivity | Patience    | Typical order behavior               | Best response                               |
|----------------------|--------------------------------|-------------------|-------------|--------------------------------------|---------------------------------------------|
| Office Worker        | Speed and predictability       | Medium-high       | Low         | Simple lunch, limited time           | Fast menu, quick seating, efficient service |
| Affluent Couple      | Experience and quality         | Low               | Medium      | Premium meals, desserts, drinks      | High-quality dishes, ambiance, good pacing  |
| Event Fan            | Portion size and convenience   | Medium            | Low-medium  | Groups, comfort food, beverages      | High throughput, group-friendly menu        |
| Tourist              | Discoverability and experience | Medium            | Medium      | Signature dishes, higher uncertainty | Visible reputation, clear menu, novelty     |
| Neighborhood Regular | Reliability and value          | Medium            | Medium-high | Familiar items, repeat tendency      | Consistency, fair pricing, satisfaction     |

Each customer or party receives a hidden preference profile. The player sees broad market information and signals, but not exact individual math.

Example customer profile:

```
{
  "segment": "office_worker",
  "budget": 18,
  "patienceSeconds": 55,
  "preferredTags": ["quick", "light", "coffee"],
  "dislikedTags": ["slow", "messy"],
  "partySize": 1,
  "serviceSpeedWeight": 0.45,
  "priceWeight": 0.30,
  "menuFitWeight": 0.15,
  "reputationWeight": 0.10
}
```

## Restaurant choice model

When a new customer party enters the shared district, it evaluates both restaurants.

A simplified initial model:

$$\text{Restaurant Appeal} = w_m(\text{menu fit}) + w_p(\text{price value}) + w_s(\text{projected service speed}) + w_r(\text{reputation})$$

The customer then chooses probabilistically rather than deterministically. This avoids a snowball where one restaurant captures every customer due to a tiny early advantage.



## Important design rule

The competitor must matter, but the player must still have agency.

A player should not lose simply because the opponent had a better starting menu. Customers must continue to react to:

- Actual queue length
- Actual service speed
- Visible reputation
- Capacity availability
- Prices
- Event changes
- Dish availability
- Temporary promotions or failures

## 7. Setup phase

### Purpose

The setup phase is where players convert market knowledge into a strategy. It should feel meaningful, but it should not become a slow spreadsheet game.

The player receives:

- Market name and district description
- Daypart
- Nearby business/event anchors
- Customer segment forecast
- Broad spending and patience indicators
- Initial event forecast, if any
- Starting cash
- Starting restaurant layout
- Initial worker roster
- Available dish catalog
- Available ingredient inventory
- Available early upgrades

The player configures:

1. Menu
2. Prices

3. Starting inventory allocation
4. Optional cuisine/service emphasis
5. Worker station assignments
6. Initial upgrade purchase
7. Optional restaurant policy/perk

## Menu configuration

Each menu slot contains a dish. A dish has:

- Category: entrée, side, drink, dessert, snack
- Ingredient requirements
- Prep time
- Cook time
- Plate time
- Base cost
- Suggested price
- Profit margin
- Difficulty
- Customer-tag affinity
- Event affinity
- Satisfaction potential
- Waste/spoilage risk
- Station requirements

Example MVP dish catalog:

| Dish             | Tags                         | Cost | Suggested price | Time   | Strength                      | Weakness                     |
|------------------|------------------------------|------|-----------------|--------|-------------------------------|------------------------------|
| Smash Burger     | hearty, fast, stadium        | \$5  | \$14            | Medium | Broad appeal, good fan fit    | Lower premium potential      |
| Caesar Salad     | light, quick, office         | \$3  | \$12            | Fast   | Strong lunch throughput       | Weak during stadium rush     |
| Pasta Primavera  | premium, theater, vegetarian | \$6  | \$22            | Medium | High satisfaction in Uptown   | Needs timing and ingredients |
| Chicken Sandwich | fast, casual, portable       | \$4  | \$13            | Fast   | Reliable lunch/stadium option | Lower differentiation        |
| Steak Frites     | premium, date-night          | \$10 | \$34            | Slow   | High margin and prestige      | Risky under high throughput  |

| Dish       | Tags                          | Cost | Suggested price | Time            | Strength           | Weakness                            |
|------------|-------------------------------|------|-----------------|-----------------|--------------------|-------------------------------------|
| Nachos     | shareable, stadium, nightlife | \$4  | \$16            | Fast            | Great for groups   | Table turnover may slow             |
| Espresso   | quick, office, premium        | \$1  | \$5             | Very fast       | Excellent add-on   | Low standalone revenue              |
| Cheesecake | dessert, premium, date-night  | \$3  | \$10            | Low active prep | High-margin add-on | Less useful in rush-only strategies |

## Menu constraints

For the MVP:

- Players choose 3 main dishes.
- Players choose up to 2 add-ons: drinks, desserts, or sides.
- Each restaurant starts with a small, fixed kitchen layout.
- Every dish must be physically producible by a station in the restaurant.
- Players can alter the menu only during setup in MVP.
- Later versions may allow limited mid-match specials, but this should not be added until baseline pacing is proven.

## Pricing

Players set a price for each selected menu item within a bounded range.

Pricing affects:

- Customer acquisition
- Customer satisfaction
- Per-order profit
- Customer segment match
- Event conversion
- Perceived value
- Potential tips
- Rival counterplay

Example pricing behavior:

- A low price improves conversion with price-sensitive diners but reduces revenue per seat.
- A high price increases potential margin but may reduce selection probability, especially if projected wait time is high.

- A premium price can work well for an affluent pre-theater crowd if service quality remains high.
- High prices combined with slow service should create sharp satisfaction penalties.

The UI should display **qualitative guidance**, not exact customer utility math:

- “Excellent value”
- “Competitive”
- “Premium”
- “Likely too expensive for this market”
- “Low margin”
- “Strong margin, demand risk”

This preserves decision-making without exposing the entire simulation.

## Staffing setup

Each player begins with a small automated staff. The owner-player supplements them.

Initial worker types:

| Worker      | Primary role                                      | Limitation                                    |
|-------------|---------------------------------------------------|-----------------------------------------------|
| Cook        | Prepares dishes at kitchen stations               | Can create a prep backlog                     |
| Server      | Seats, takes orders, delivers food, clears tables | Can get overwhelmed by large dining room      |
| Prep Worker | Restocks ingredients and handles prep             | Lower priority intelligence than player       |
| Host        | Seats customers and manages waiting line          | Optional for MVP; may be abstracted initially |

For the first playable version, use:

- 1 cook
- 1 server
- 1 owner-player
- Abstract host behavior or automatic seating

Later, add specialization and worker upgrades.

## Initial policies/perks

Each player chooses one small strategic policy before service begins.

Examples:

| Policy          | Effect                                                        | Intended strategy     |
|-----------------|---------------------------------------------------------------|-----------------------|
| Express Service | Faster basic order processing, lower max satisfaction ceiling | High-throughput lunch |

| Policy             | Effect                                                 | Intended strategy        |
|--------------------|--------------------------------------------------------|--------------------------|
| House Special      | One tagged dish gets increased appeal and satisfaction | Market specialization    |
| Friendly Staff     | Satisfaction decays more slowly during waits           | Consistency/recovery     |
| Lean Operations    | Lower ingredient cost, slower restocking               | Margin-focused play      |
| Premium Experience | Higher tip potential, slower table turnover            | High-end dinner strategy |

For MVP, either omit policies or include only two to reduce balance complexity.

## 8. Service phase

### Player avatar

The player controls the restaurant owner as a visible in-world character.

### Controls

Recommended first implementation:

- WASD: move
- Mouse movement: rotate camera / aim contextual direction
- E: contextual interact
- F: secondary action or quick service action
- Shift: short sprint, limited by stamina or cooldown
- 1-4: optional quick task/priority commands in later versions
- Tab: tactical overview panel
- Esc: pause/settings in single-player harness only; no pause in multiplayer

For an isometric camera variant:

- Click-to-move is acceptable, but direct WASD movement better supports the “actively run the restaurant” fantasy.
- Keep camera rotation limited or fixed in the initial version to preserve readability.

### Interactions

The owner can:

- Pick up raw ingredients
- Restock a kitchen station
- Prepare a dish faster than an automated worker
- Plate a finished dish
- Carry a plate to a table

- Clear tables
- Seat a waiting party
- Refill table items
- Repair/unstick a station
- Handle a dissatisfied customer
- Purchase an upgrade from a physical office/terminal location
- Trigger limited tactical boosts in later builds

The interaction system should be contextual. When near a valid object, display a brief prompt such as:

- E – Cook Smash Burger
- E – Deliver Order #12
- E – Restock Lettuce
- E – Buy Faster Grill: \$250

### Operational bottlenecks

The game should create solvable but overlapping bottlenecks.

| Bottleneck          | Visible signal                      | Player intervention              | Consequence if ignored                  |
|---------------------|-------------------------------------|----------------------------------|-----------------------------------------|
| Kitchen backlog     | Orders queue at station             | Cook/plate items personally      | Slow service and reduced satisfaction   |
| Ingredient shortage | Empty ingredient icon               | Retrieve/restock ingredients     | Menu items unavailable, canceled orders |
| Server overload     | Food waiting at pass, unused tables | Deliver, clear, seat customers   | Tables remain blocked, customers wait   |
| Long entry queue    | Customers outside/at host stand     | Seat parties or improve capacity | Customers choose rival or leave         |
| Unhappy customer    | Red patience meter/body language    | Deliver, apologize, comp item    | Satisfaction and reputation loss        |
| Dirty table         | Visible table state                 | Clear and reset table            | Reduced seating capacity                |
| Equipment failure   | Warning icon/steam/sparks           | Repair or work around it         | Slower production                       |
| Cash opportunity    | Upgrade terminal callout            | Travel and purchase upgrade      | Missed advantage if delayed             |

The player should nearly always have more than one useful task. Skill comes from prioritization, pathing, anticipation, and choosing when to let automation handle something.

## Customer state machine

Every customer party follows a state model:

```
ENTER_DISTRICT
-> EVALUATE_RESTAURANTS
-> APPROACH_OR_QUEUE
-> SEATED
-> ORDERING
-> WAITING_FOR_FOOD
-> EATING
-> PAYING
-> LEAVING
-> REVIEW/REPUTATION_IMPACT
```

Exit states include:

```
CHOOSE_RIVAL
LEAVE_DISTRICT
ABANDON_QUEUE
CANCEL_ORDER
LEAVE_ANGRY
```

A customer's satisfaction should be affected by:

- Wait to be seated
- Wait to order
- Wait for food
- Dish quality
- Dish-to-preference match
- Price perceived as fair/unfair
- Order accuracy
- Table cleanliness
- Restaurant ambiance score
- Event relevance
- Recovery actions
- Total visit duration relative to their patience profile

Restaurant simulations commonly connect service speed, correct fulfillment, tips/reviews, and future traffic, which supports making wait time and satisfaction central to the scoring and reputation loop.<sup>[5]</sup>

# 9. Dynamic events

## Event cadence

Random or scheduled events are announced every 30–60 seconds during the service phase.

The event system should use a seeded event deck rather than fully unconstrained randomness. This allows:

- Fair multiplayer conditions
- Replays/debugging
- Balance testing
- Match determinism
- Event forecasting in setup
- Prevention of impossible/unfun combinations

Each match receives the same event timeline for both players. Events may affect players asymmetrically only because their menus, prices, upgrades, and current restaurant states differ.

## Event announcement flow

1. Event teaser or forecast appears 10–20 seconds before activation when appropriate.
2. Main announcement appears with icon, short description, and countdown.
3. Event activates.
4. District/customer behavior updates.
5. Event ends or transitions to the next event.

Example UI:

BASEBALL GAME ENDS IN 20 SECONDS

Expect a large surge of hungry groups. Shareable dishes and quick service are in demand.

## MVP events

| Event                | Duration | Main effect                                        | Strategic response                        |
|----------------------|----------|----------------------------------------------------|-------------------------------------------|
| Office Break Starts  | 45 sec   | More office workers; lower patience                | Fast dishes, seating, quick delivery      |
| Theater Curtain Call | 60 sec   | Upscale demand spike; premium dishes favored       | Prioritize quality and high-margin meals  |
| Baseball Game Ends   | 60 sec   | High-volume groups; hearty/shareable foods favored | Throughput, group seating, beverage sales |
| Rainstorm            | 45 sec   | Foot traffic briefly slows, dine-in patience rises | Use time for upgrades, prep, cleanup      |



| Event                 | Duration | Main effect                                       | Strategic response                               |
|-----------------------|----------|---------------------------------------------------|--------------------------------------------------|
| Food Critic Spotted   | 45 sec   | One high-value critic party enters                | Deliver perfect service to gain reputation       |
| Ingredient Shortage   | 30 sec   | One ingredient restocks more slowly or costs more | Shift production priorities, avoid affected dish |
| Local Influencer Post | 60 sec   | Trendy/signature dish demand increases            | Push signature menu item                         |
| Transit Delay         | 45 sec   | Customer arrivals shift later, burst queues form  | Prepare capacity and avoid idle labor            |
| Happy Hour Nearby     | 60 sec   | Price-sensitive nightlife traffic increases       | Value menu, quick drinks, lower price points     |
| Power Fluctuation     | 30 sec   | Selected equipment slows                          | Manual intervention and contingency routing      |

## Event design rules

- Events must create an actionable decision, not merely change a number.
- Event effects should be explained in plain language.
- Do not stack more than two high-impact events at once in MVP.
- Avoid pure negative events that feel unavoidable and random.
- Give players a warning when an event can reasonably be anticipated.
- Make some events favor specific menus, while others favor operational quality.

# 10. Upgrades

## Upgrade philosophy

Upgrades are purchased during service by physically moving the owner to an in-world management terminal, office, storage room, or equipment location.

This requirement is important: upgrades should force a decision. Spending cash must cost time and attention.

## Upgrade categories

| Category            | Example upgrade     | Effect                                 |
|---------------------|---------------------|----------------------------------------|
| Kitchen speed       | Faster Grill        | Reduces grill cook duration            |
| Prep throughput     | Prep Counter        | Increases concurrent prep capacity     |
| Delivery            | Serving Tray        | Owner can carry two dishes             |
| Staff efficiency    | Server Radio        | Server target selection improves       |
| Customer experience | Comfortable Seating | Slower satisfaction decay while seated |

| Category              | Example upgrade      | Effect                                                      |
|-----------------------|----------------------|-------------------------------------------------------------|
| Capacity              | Additional Table     | Adds seating, but increases server load                     |
| Restocking            | Organized Pantry     | Faster owner/staff ingredient retrieval                     |
| Reputation            | Street Signage       | Improves restaurant consideration among undecided customers |
| Recovery              | Complimentary Snacks | Enables faster dissatisfaction recovery                     |
| Equipment reliability | Maintenance Plan     | Reduces station failure chance                              |

## Upgrade tiers

For MVP:

- Three upgrade tiers maximum per category.
- Use clearly visible physical changes where possible.
- Avoid exponential upgrade chains.
- Prioritize upgrades that alter player decisions or spatial flow, rather than only increasing a scalar value.

Example:

| Upgrade         | Cost  | Effect                   | Why it matters                              |
|-----------------|-------|--------------------------|---------------------------------------------|
| Serving Tray I  | \$125 | Owner carries 2 plates   | Makes owner delivery runs efficient         |
| Serving Tray II | \$275 | Owner carries 3 plates   | Strong during group-rush events             |
| Faster Grill I  | \$180 | Grill cook time -15%     | Increases burger/stadium strategy viability |
| Better Seating  | \$200 | Seated patience +15%     | Gives server more room to recover           |
| Pantry Shelves  | \$150 | Restock travel time -25% | Reduces ingredient shortage disruption      |

# 11. Scoring and win conditions

## Primary win condition

At the end of the match, the restaurant with the highest **Restaurant Score** wins.

The score should not be raw revenue alone. A pure revenue score encourages degenerate price strategies and ignores the service-management fantasy.

## Recommended score composition

[  
**Restaurant Score =**  
**Revenue Score +**  
**Guests Served Score +**  
**Satisfaction Score +**  
**Reputation Bonus +**  
**Event Objective Bonus**  
 Penalty Score  
 ]

Suggested MVP weighting:

| Metric               | Weight / calculation | Rationale                            |
|----------------------|----------------------|--------------------------------------|
| Net revenue          | 40%                  | Rewards profitable strategy          |
| Customers served     | 20%                  | Rewards throughput and capacity      |
| Average satisfaction | 25%                  | Rewards quality and recovery         |
| Reputation           | 10%                  | Rewards consistent guest experience  |
| Event objectives     | 5%                   | Rewards reacting to match conditions |

## Penalties

- Customer abandonment
- Canceled orders
- Severe dissatisfaction
- Unserved food waste
- Broken promises such as failing a critic event
- Extreme pricing mismatch, if applicable

## Tie-breakers

1. Higher average satisfaction
2. More customers served
3. Higher net revenue
4. Fewer abandoned parties

## End-of-match results

The results screen should explain *why* a player won or lost.

Display:

- Winner and final score
- Revenue
- Expenses
- Net profit
- Customers served
- Customers lost to rival
- Average satisfaction
- Average wait time
- Best-selling dishes
- Highest-margin dishes
- Event performance
- Upgrades purchased
- Customer-segment breakdown
- Key turning points

Example narrative insight:

You won the lunch rush by serving 18 more office-worker parties. Your Caesar Salad had the fastest average fulfillment time, while your rival lost 11 customers to queue abandonment after the transit-delay event.

This explanation layer is important because it turns each match into a learning loop rather than a black-box simulation.

## 12. Multiplayer design

### Mode

Initial release supports:

- Private 1v1 rooms
- Quick Match 1v1, if a basic queue is feasible
- Developer local multiplayer for testing
- Bot opponent fallback for solo play and development

## Room flow

1. Player creates or joins a room.
2. Both players connect through WebSockets.
3. Server assigns a match seed.
4. Server generates or selects the market scenario.
5. Both clients receive identical public market data.
6. Both clients complete setup independently.
7. Server begins service phase once both players are ready or timer expires.
8. Server runs authoritative simulation ticks.
9. Clients send player input/action intents.
10. Server validates actions and broadcasts state snapshots/deltas.
11. Server calculates final score and returns match results.

## Networking model

Use an **authoritative server**. The browser must not be trusted to calculate money, customer choices, scores, inventory, upgrades, or action outcomes.

## Server authority

The server owns:

- Match seed
- Match timer
- Event timeline
- Customer spawning
- Customer restaurant selection
- Prices and menu validity
- Restaurant inventory
- Worker state
- Customer satisfaction
- Order state
- Revenue and cost calculations
- Upgrade purchase validation
- Player interaction validation
- Match scoring
- Results

## Client responsibilities

The client owns:

- Rendering
- Input capture
- Camera movement
- UI state
- Visual interpolation
- Local sound and VFX
- Non-authoritative prediction for player movement, if needed
- Developer scene harnesses

## Tick target

For MVP:

- Server simulation: 10–20 ticks per second
- Network state broadcast: 10 ticks per second initially
- Client rendering: `browser requestAnimationFrame`
- Use interpolation between snapshots for smooth movement

A lower authoritative tick rate is acceptable for restaurant operations because most objects move slowly and interactions are discrete. Do not over-engineer rollback networking for the first version.

## WebSocket messages

Use JSON messages initially for speed of development and debuggability.

## Client-to-server

```
{
  "type": "player_input",
  "sequence": 482,
  "move": {
    "x": 0.0,
    "z": 1.0,
    "sprint": false
  },
  "facing": 1.57
}
```

```
{
  "type": "interact",
  "sequence": 483,
```

```
"targetId": "station_grill_1",
"action": "cook"
}
```

```
{
  "type": "purchase_upgrade",
  "sequence": 484,
  "upgradeId": "faster_grill_1"
}
```

```
{
  "type": "setup_submit",
  "menu": [
    { "dishId": "smash_burger", "price": 14 },
    { "dishId": "chicken_sandwich", "price": 13 },
    { "dishId": "nachos", "price": 16 }
  ],
  "addons": [
    { "dishId": "espresso", "price": 5 }
  ],
  "startingUpgradeId": "serving_tray_1",
  "staffAssignments": {
    "cook_1": "grill",
    "server_1": "dining_room"
  }
}
```

## Server-to-client

```
{
  "type": "match_snapshot",
  "serverTime": 154320,
  "matchPhase": "service",
  "timeRemainingMs": 322000,
  "events": [
    {
      "eventId": "baseball_game_ends",
      "state": "warning",
      "startsInMs": 18000
    }
  ],
  "restaurants": [],
  "customers": [],
  "orders": [],
  "players": []
}
```

```
{
  "type": "event_announce",
  "eventId": "food_critic_spotted",
  "title": "Food Critic Spotted",
}
```

```
"description": "A high-value critic party is approaching. Exceptional service can b
"startsInMs": 0,
"durationMs": 45000
}
```

```
{
  "type": "match_complete",
  "winnerPlayerId": "player_1",
  "results": {
    "player_1": {},
    "player_2": {}
  }
}
```

## 13. Architecture

### Repository layout

```
rival-restaurant/
├─ README.md
├─ package.json
├─ shared/
│   └─ game-data/
│       ├── dishes.json
│       ├── markets.json
│       ├── events.json
│       ├── upgrades.json
│       └─ customer-segments.json
│   └─ schemas/
│       ├── messages.ts
│       ├── game-state.ts
│       └─ validation.ts
│   └─ constants/
│       └─ tuning.ts
├─ server/
│   ├── package.json
│   └─ src/
│       ├── index.js
│       ├── http/
│       │   ├── routes.js
│       │   └─ health.js
│       ├── websocket/
│       │   ├── socket-server.js
│       │   ├── connection-manager.js
│       │   └─ message-router.js
│       ├── game/
│       │   ├── match-manager.js
│       │   ├── match.js
│       │   ├── simulation-loop.js
│       │   └─ systems/
│       │       ├── customer-system.js
│       │       └─ order-system.js
```



```
|
|
|
|
|   ├── worker-system.js
|   ├── restaurant-system.js
|   ├── event-system.js
|   ├── scoring-system.js
|   └── movement-system.js
|   └── validators/
|       ├── action-validator.js
|       └── setup-validator.js
|   └── persistence/
|       └── in-memory-store.js
|   └── public/
|       └── client-build/
└── client/
    ├── package.json
    ├── index.html
    ├── src/
    │   ├── main.tsx
    │   ├── app/
    │   ├── ui/
    │   └── game/
    │       ├── GameClient.ts
    │       ├── NetworkClient.ts
    │       ├── InputController.ts
    │       ├── SceneManager.ts
    │       ├── EntityViewRegistry.ts
    │       ├── CameraController.ts
    │       ├── InteractionController.ts
    │       └── StateInterpolator.ts
    │   └── scenes/
    │       ├── RestaurantScene.ts
    │       ├── DistrictScene.ts
    │       └── ResultsScene.ts
    │   └── styles/
    └── vite.config.ts
└── harnesses/
    ├── package.json
    ├── index.html
    ├── src/
    │   ├── harness-shell.ts
    │   ├── restaurant-layout-harness.ts
    │   ├── customer-flow-harness.ts
    │   ├── kitchen-bottleneck-harness.ts
    │   ├── event-visualization-harness.ts
    │   ├── upgrade-preview-harness.ts
    │   └── shared/
    │       ├── scene-primitives.ts
    │       ├── test-entities.ts
    │       └── dev-controls.ts
    └── vite.config.ts
└── assets/
    ├── models/
    ├── textures/
    ├── audio/
    └── licenses/
```

This is intentionally a simple multi-folder repository, not a monorepo framework. Root scripts may coordinate local development, but each application remains independently understandable and runnable.

## Client stack

### Required technologies

- TypeScript
- React
- Three.js loaded from CDN
- Vite only as a lightweight client/harness development and build tool
- Plain CSS, CSS modules, or a small styling approach; avoid introducing a large UI system early
- Native browser WebSocket API

### Three.js loading

Use a pinned import map in `index.html`. Do not rely on an unversioned “latest” URL.

```
<script type="importmap">
{
  "imports": {
    "three": "https://cdn.jsdelivr.net/npm/three@0.180.0/build/three.module.js",
    "three/addons/": "https://cdn.jsdelivr.net/npm/three@0.180.0/examples/jsm/"
  }
}
</script>
```

Then:

```
import * as THREE from "three";
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";
```

The exact Three.js version should be pinned in one shared project constant and updated deliberately. Browser imports need an import map to resolve the `three` specifier used by Three.js addons, and mixing CDN/version sources can break module compatibility. <sup>[2]</sup> <sup>[1]</sup>

### React responsibilities

React should manage:

- Lobby
- Setup screen
- Market briefing

- Menu and pricing controls
- Upgrade panel
- HUD
- Event announcements
- Results screen
- Debug controls in development
- Accessibility overlays and non-3D state presentation

React should **not** attempt to reconcile every Three.js entity as JSX state every simulation tick. Three.js owns high-frequency scene rendering; React owns application UI and lower-frequency game state panels.

## Server stack

### Required technologies

- Node.js
- JavaScript
- Express
- ws
- Native Node utilities where possible
- In-memory storage for MVP
- Optional SQLite or Postgres only after persistent profiles/match history are required

### Server responsibilities

- Serve the built client application
- Expose health endpoint
- Manage game rooms
- Manage WebSocket connections
- Run authoritative matches
- Validate all client actions
- Generate deterministic market/event seeds
- Broadcast state updates
- Handle reconnect grace period
- Return results and telemetry

### Suggested HTTP endpoints

| Endpoint           | Method | Purpose                                   |
|--------------------|--------|-------------------------------------------|
| /health            | GET    | Deployment and service health             |
| /api/version       | GET    | Build/client compatibility                |
| /api/markets       | GET    | Development/debug market definitions      |
| /api/rooms         | POST   | Create private room                       |
| /api/rooms/:roomId | GET    | Basic room status                         |
| /api/dev/match     | POST   | Development-only bot/local match creation |

The actual game session should use WebSockets, not repeated REST polling.

## 14. 3D scene design

### Camera

Recommended initial presentation:

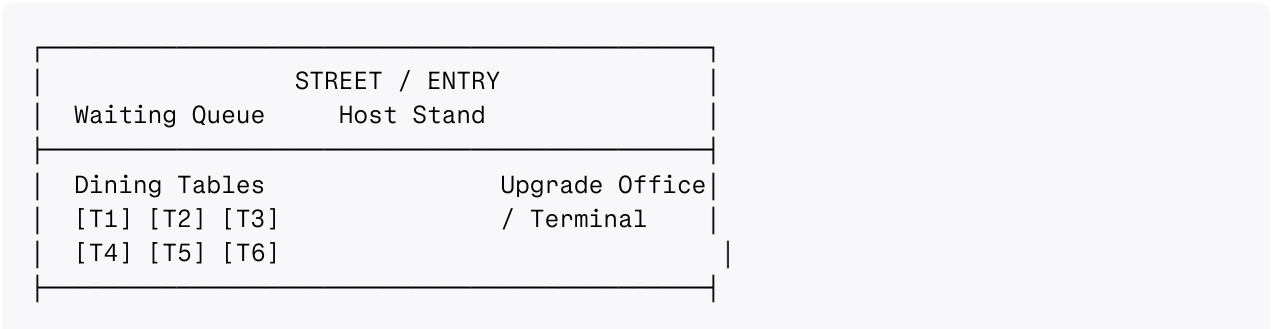
- High-angle third-person/isometric camera
- Player restaurant occupies most of the screen
- Rival restaurant is visible across a street, behind glass, or through a simplified cutaway
- Camera follows player owner with soft bounds
- Optional strategic overhead view via Tab
- Minimal camera collision complexity in MVP

### Why this approach

A fully free third-person camera may look impressive but makes restaurant state hard to read. A high-angle camera preserves character movement while allowing the player to see queues, tables, kitchen stations, and staff.

### Restaurant layout

Initial restaurant should be compact and highly readable.



|                                                                                            |
|--------------------------------------------------------------------------------------------|
| Service Pass / Pickup Counter                                                              |
| Prep Station   Grill   Oven   Plating<br>Storage / Pantry                      Dishwashing |

The owner's travel path must matter, but the space must remain small enough that physical movement does not become a tedious commute.

## MVP entities

| Entity           | Visual simplification                         |
|------------------|-----------------------------------------------|
| Owner player     | Low-poly stylized humanoid                    |
| Workers          | Color-coded humanoids or silhouettes          |
| Customers        | Segment-specific color/style cues             |
| Tables           | Simple table/chair sets with state badges     |
| Kitchen stations | Clear color/material distinction per station  |
| Ingredients      | Crates, bins, shelves, icons                  |
| Food             | Simple recognizable plated low-poly items     |
| Upgrade terminal | Office desk/tablet/cash register visual       |
| Rival restaurant | Simplified but visibly active mirrored layout |

## Visual state language

Use consistent signals:

- Green: healthy/ready
- Yellow: attention needed soon
- Orange: active bottleneck
- Red: critical / abandonment risk
- Blue: customer/event opportunity
- Purple: premium/high-value opportunity

Examples:

- Customer patience ring below feet
- Order ticket above table
- Station queue indicator
- Ingredient shortage icon
- Worker role icon
- Food-ready icon at service pass

- Floating cash/tip feedback only for major moments, not every transaction
- Event banner at top center with a district-level visual effect

## 15. Developer harnesses

### Goal

Harnesses are standalone, focused 3D scenes that make visual/gameplay iteration cheap. They must use the same reusable scene/entity/view components as the actual game where practical, but they should not require:

- A live backend
- A full multiplayer room
- Authentication
- The full React game shell
- All gameplay systems to be running

A designer/developer should be able to launch a harness, alter a scene configuration, and immediately inspect the result.

### Required MVP harnesses

#### 15.1 Restaurant layout harness

Purpose:

- Test restaurant footprint
- Test camera framing
- Test object placement
- Test navigation space
- Validate line of sight and readability

Controls:

- Toggle kitchen/dining-room layouts
- Toggle day/night lighting
- Toggle competitor visibility
- Enable debug grid
- Spawn owner and staff
- Adjust camera height, angle, and zoom
- Test blocked path locations

## 15.2 Customer flow harness

Purpose:

- Visualize customers entering, choosing a restaurant, queuing, seating, eating, paying, and leaving
- Tune pathing and crowd density
- Validate that customer states are understandable

Controls:

- Spawn customer segment
- Change party size
- Force player restaurant/rival/abandon decisions
- Adjust patience
- Simulate queue size
- Toggle route lines and state labels

## 15.3 Kitchen bottleneck harness

Purpose:

- Test station queues, ingredient shortages, finished-food pickup, worker behaviors, and owner interventions

Controls:

- Queue preset orders
- Disable a worker
- Empty an ingredient bin
- Slow a station
- Trigger equipment failure
- Toggle production speed
- Spawn ready dishes at service pass
- Measure task completion time

## 15.4 Event visualization harness

Purpose:

- Test event announcements and environmental changes

Controls:

- Trigger each event
- Preview warning/active/ending states

- Simulate foot-traffic changes
- Toggle district props, lighting, crowd density, or weather
- Inspect event UI at multiple resolutions

## 15.5 Upgrade preview harness

Purpose:

- Ensure purchases create visible, satisfying physical changes

Controls:

- Toggle each upgrade tier
- Compare before/after station visuals
- Preview capacity additions
- Preview performance overlays
- Test upgrade terminal interaction range

## Harness interface

A generic harness contract could look like:

```
export interface SceneHarness {  
  id: string;  
  title: string;  
  description: string;  
  mount(container: HTMLElement): void;  
  dispose(): void;  
}
```

Example harness registration:

```
import { restaurantLayoutHarness } from "../restaurant-layout-harness";  
import { customerFlowHarness } from "../customer-flow-harness";  
import { kitchenBottleneckHarness } from "../kitchen-bottleneck-harness";  
  
export const harnesses = [  
  restaurantLayoutHarness,  
  customerFlowHarness,  
  kitchenBottleneckHarness  
];
```

The key requirement is separation: game rules should emit state, and scene-view code should render state. A harness can then provide mocked state without simulating an entire match.



## 16. Data-driven content

All balancing content should be JSON or plain data modules rather than hardcoded in systems.

### Market definition example

```
{
  "id": "downtown_lunch",
  "name": "Downtown Lunch Rush",
  "daypart": "lunch",
  "description": "Office towers, transit stops, and a tightly timed weekday crowd.",
  "segmentWeights": {
    "office_worker": 0.55,
    "tourist": 0.10,
    "neighborhood_regular": 0.20,
    "affluent_couple": 0.05,
    "event_fan": 0.10
  },
  "priceSensitivity": 1.2,
  "baseFootTrafficPerMinute": 14,
  "preferredTags": ["quick", "light", "portable"],
  "eventPool": [
    "office_break_starts",
    "transit_delay",
    "rainstorm",
    "food_critic_spotted"
  ]
}
```

### Dish definition example

```
{
  "id": "smash_burger",
  "name": "Smash Burger",
  "category": "entree",
  "tags": ["hearty", "fast", "stadium", "casual"],
  "ingredients": {
    "bun": 1,
    "beef": 1,
    "cheese": 1,
    "lettuce": 1
  },
  "baseCost": 5,
  "suggestedPrice": 14,
  "stationSteps": [
    { "station": "prep", "durationMs": 2500 },
    { "station": "grill", "durationMs": 6000 },
    { "station": "plating", "durationMs": 1500 }
  ],
  "baseSatisfaction": 65,
  "marketAffinity": {
    "stadium_district": 1.25,
    "downtown_lunch": 1.05,
  }
}
```

```
    "uptown_pre_theater": 0.70
  }
}
```

## Event definition example

```
{
  "id": "baseball_game_ends",
  "title": "Baseball Game Ends",
  "warningMs": 15000,
  "durationMs": 60000,
  "effects": {
    "footTrafficMultiplier": 1.8,
    "segmentWeightOverrides": {
      "event_fan": 0.65,
      "tourist": 0.20
    },
    "dishTagDemandMultipliers": {
      "stadium": 1.35,
      "shareable": 1.25,
      "hearty": 1.15
    },
    "partySizeMultiplier": 1.4
  }
}
```

# 17. Core systems specification

## Customer acquisition system

For each incoming party:

1. Determine customer segment and party size.
2. Calculate current district conditions.
3. Evaluate both restaurants using public restaurant properties and current visible operational state.
4. Add controlled randomness.
5. Send party to:
  - Player restaurant
  - Rival restaurant
  - Neither restaurant / district exit
6. Record decision reason for analytics and post-match explanation.

Track decision reasons such as:

- `better_price`

- better\_menu\_fit
- shorter\_projected\_wait
- higher\_reputation
- event\_affinity
- restaurant\_full
- customer\_abandoned\_queue

This will be invaluable for balance and player-facing end-of-match insights.

## Order system

A selected customer party:

1. Gets seated.
2. Generates an order based on segment preferences, menu availability, price, and event context.
3. Adds order tickets to kitchen workflow.
4. Waits while patience declines.
5. Receives food.
6. Evaluates food quality, price value, and wait time.
7. Eats and pays.
8. Modifies restaurant score/reputation.
9. Leaves, freeing a table.

## Order quality

Order quality should be a combination of:

- Correctness: required dish delivered
- Freshness: time since completion
- Preparation quality: worker/player execution tier in later versions
- Customer preference fit
- Ingredient quality or upgrades in later versions
- Service timing

For MVP, do not add complex minigames for cooking. The meaningful skill should be operational prioritization, physical movement, and timing—not an unrelated button-sequence challenge.

## Worker AI system

Workers need simple, explainable rules.

Example cook priority:

1. Continue current task if near completion.
2. Complete highest urgency ticket at assigned station.
3. Prefer order with highest patience risk.
4. If no order exists, perform low-priority prep/restock.
5. If blocked, emit a visible “needs help” signal.

Example server priority:

1. Deliver food that is ready.
2. Seat waiting party if table is available.
3. Take order from newly seated party.
4. Clear dirty table.
5. Handle payment.
6. Idle near service area.

The owner-player should outperform workers in speed/flexibility but should not make workers irrelevant. A strong player amplifies automation rather than replacing every job permanently.

# 18. UX requirements

## HUD

The HUD must provide immediate operational awareness without overwhelming the screen.

Required elements:

- Match timer
- Current score comparison
- Revenue and available cash
- Customer count / queue warning
- Average satisfaction
- Current active event
- Upcoming event warning
- Critical alerts
- Owner carry inventory
- Selected contextual interaction
- Upgrade availability indicator

- Compact rival summary:
  - Rival score
  - Rival customer count
  - Rival satisfaction trend
  - Optional visible “busy/overloaded” state

## Alert prioritization

Limit critical alerts to prevent alarm fatigue.

Priority order:

1. Customer abandonment imminent
2. Food ready but undelivered
3. Ingredient shortage blocks active order
4. Equipment problem
5. Event countdown
6. Upgrade available
7. General operational suggestions

## Setup UI

The setup interface should be highly readable and quick.

Recommended layout:

- Left: market briefing and customer forecast
- Center: menu slots and dish options
- Right: prices, margins, starting resources, readiness
- Bottom: staff assignments and upgrade/perk selection
- Top: countdown clock and opponent-ready status

Do not reveal the opponent’s exact menu or prices during setup. Optionally reveal broad public restaurant positioning after setup, such as:

- “Rival is running a premium menu”
- “Rival is emphasizing fast service”
- “Rival has a high average price”

For MVP, it is also acceptable to reveal nothing until service begins.

# 19. Art and audio direction

## Art direction

Recommended style: stylized low-poly diorama restaurant management.

Reasons:

- Achievable with a small team
- Reads clearly from an elevated camera
- Works well with reusable/open models
- Allows exaggerated, readable customer animations
- Avoids photorealistic asset scope
- Gives upgrades visible personality

## Visual references in principle

- Clean, colorful simulation readability
- Slightly exaggerated, toy-like restaurant props
- Strong state colors and silhouettes
- Distinct district palettes:
  - Downtown: cool gray, blue, bright signage
  - Uptown: warm gold, red velvet, refined lighting
  - Stadium: team-color lights, concrete, banners
  - Nightlife: neon accents, purple/blue
  - Neighborhood: warm, natural, inviting

## Audio direction

MVP audio:

- Ambient district noise
- Restaurant crowd level that reflects occupancy
- Kitchen station sounds
- Bell/notification when food is ready
- Customer impatience cues
- Cash/tip feedback
- Event stinger
- Upgrade purchase feedback
- Results sting

Audio should reinforce stress and recovery. During a surge, crowd and kitchen intensity should rise; when the restaurant is controlled, the space should feel smoother and rewarding.

## 20. MVP scope

### In scope

- Browser desktop build
- Private 1v1 multiplayer rooms
- Bot opponent for local testing
- One compact restaurant layout
- Three market scenarios
- Five customer segments
- Eight to ten dishes
- Three to five random events
- Setup phase with menu and price selection
- Owner movement and contextual interactions
- One cook and one server per restaurant
- Basic worker behavior
- Basic inventory/restocking
- Basic kitchen and delivery workflow
- Five to eight upgrades
- Authoritative Express + WebSocket server
- Full results screen
- Restaurant layout, customer flow, and kitchen bottleneck harnesses
- Placeholder or legally reusable 3D models
- Seeded match/event generation
- Basic telemetry/logging

### Explicitly out of scope for MVP

- Four-player multiplayer
- Persistent progression
- Cosmetics store
- Matchmaking ranking/MMR
- Voice chat
- User accounts

- Complex worker hiring/firing
- Deep restaurant building/editor mode
- Dynamic menu changes during service
- Dozens of cuisines
- Complex cooking minigames
- Cross-platform mobile controls
- Full economic campaign
- AI-generated assets in runtime
- Advanced anti-cheat beyond server authority and validation
- Database-backed historical player profiles
- Spectator mode
- Replays, except for server event logs/debug traces

## 21. Milestones

### Milestone 0: Technical spike

Goal: prove the technical architecture.

Deliverables:

- Express server serving a static client
- ws WebSocket connection
- Two browser clients connected to one room
- Basic 3D restaurant scene via Three.js CDN import map
- Two owner-player avatars moving in replicated state
- Restaurant layout harness
- Deterministic seeded match configuration
- Basic dev logs

Success criteria:

- Two clients see each other move consistently.
- Three.js is not bundled in the client build.
- The server is authoritative over position bounds and basic interactions.
- A standalone harness launches without a live match.



## **Milestone 1: Single-restaurant vertical slice**

Goal: prove the active restaurant loop before competitive logic.

Deliverables:

- One restaurant
- One owner player
- One cook and one server
- Customers enter, queue, sit, order, wait, eat, pay, leave
- One dish production chain
- One ingredient shortage
- Table cleanup
- Satisfaction and money
- Kitchen bottleneck harness
- Basic HUD

Success criteria:

- The player has meaningful choices about where to help.
- A player can recover a struggling shift by intervening effectively.
- The player can clearly identify why a customer is unhappy.

## **Milestone 2: Competitive shared market**

Goal: prove that both restaurants meaningfully compete.

Deliverables:

- Two restaurants in one district
- Shared customer-spawn system
- Customer restaurant-choice model
- Setup menu/pricing
- Score comparison
- At least three customer segments
- One market scenario
- Multiplayer authoritative match lifecycle
- Results screen with reasons for customer choice

Success criteria:

- Different menu/pricing choices visibly lead to different customer distributions.
- Players can see the impact of service delays on customer choice.

- A player can come back from a weaker setup through better execution.

### **Milestone 3: Event and upgrade depth**

Goal: create replayability and strategic adaptation.

Deliverables:

- Three market scenarios
- Five customer segments
- Five or more events
- Five or more upgrades
- Physical upgrade purchase interaction
- Event visualization harness
- Upgrade preview harness
- Improved worker priority logic
- Match analytics

Success criteria:

- Players adapt during service rather than repeating a fixed strategy.
- Events are understandable and materially change decisions.
- Upgrades create visible and tactical changes.

### **Milestone 4: Polish and closed testing**

Goal: make the game understandable, stable, and fun in repeated 1v1 play.

Deliverables:

- Onboarding/tutorial prompts
- Improved UI and visual language
- Sound effects and feedback
- Better pathing
- Reconnection handling
- Match telemetry dashboard/log export
- Balance tuning
- Asset/license audit
- 10–20 external playtests

Success criteria:

- New players can complete a match without external explanation.

- Most players understand why they lost.
- Matches consistently fit within the target session length.
- No common desync, duplicate-purchase, or invalid-action bugs.

## 22. Acceptance criteria

### Core gameplay

- Two players can join the same match and operate separate restaurants.
- Both restaurants draw customers from one shared district pool.
- Customer choice changes based on menu fit, price, projected wait, reputation, and event effects.
- Players configure a menu and prices during a timed setup phase.
- Players directly move an owner character during service.
- The owner can cook, restock, deliver, clear tables, and purchase upgrades.
- Automated workers perform basic restaurant work.
- Random/seeded events occur approximately every 30–60 seconds.
- Events visibly and mechanically affect demand, pricing tolerance, dish desirability, or restaurant operations.
- The winner is determined by a composite score, not raw money alone.
- Results explain key performance differences.

### Technical

- Frontend uses JavaScript/TypeScript, React, and Three.js.
- Three.js is imported from a pinned CDN and is not bundled into application dependencies.
- Backend is a vanilla Express JavaScript app.
- Multiplayer uses WebSockets.
- Server validates all economy-changing and score-changing actions.
- Client/server code is separated by folders.
- The repository does not rely on Next.js or a monorepo framework.
- At least three visual harnesses exist and run independently from a live multiplayer match.
- Match simulation is seedable and supports reproducible event timelines.

## Quality

- UI remains readable during a full restaurant rush.
- Important bottlenecks are visible in the 3D scene.
- A poor pre-match strategy can be partially recovered through execution.
- A good pre-match strategy can still fail through poor execution.
- No single dish, market, or upgrade should dominate across all scenarios.
- Players have meaningful decisions at least every 5–10 seconds during service.

## 23. Key risks and mitigations

| Risk                                            | Why it matters                                                                                          | Mitigation                                                                                               |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Simulation complexity grows too fast            | Customers, workers, kitchens, pricing, events, and multiplayer can compound into an unmanageable system | Build one vertical slice first; use data-driven systems; limit MVP entities and dishes                   |
| A dominant strategy emerges                     | One menu/price strategy may trivialize the game                                                         | Use market-specific demand, hidden preference variance, event shifts, and telemetry                      |
| Early snowballing                               | Winner captures customers, earns more, upgrades more, then captures even more                           | Keep customer choice probabilistic; add queue/reputation recovery; cap runaway advantages                |
| Player workload becomes exhausting              | Too many alerts/tasks can feel chaotic rather than skillful                                             | Limit restaurant size, prioritize alerts, use capable baseline workers, tune task frequency              |
| Multiplayer desync or cheating                  | Browser-authoritative economy would be exploitable                                                      | Make server authoritative for state, money, orders, and score                                            |
| Three.js integration becomes tangled with React | High-frequency game state can cause rendering/performance issues                                        | Separate React UI from imperative Three.js scene layer                                                   |
| Asset licensing issues                          | Reused models may have restrictions                                                                     | Store source/license metadata in <code>assets/licenses</code> ; prefer CC0 or commercial-friendly assets |
| Scope creep                                     | Restaurant-builder and management systems are broad                                                     | Preserve strict MVP exclusions and milestone gates                                                       |

## 24. Initial balance hypotheses

These are starting hypotheses to test, not final rules.

- One full service match should produce approximately 40–90 customer parties per restaurant depending on market.
- A player should personally intervene in 20–40 meaningful tasks per match.
- Automated staff should complete approximately 60–75% of routine work.
- The player should be responsible for the most time-sensitive 25–40% of outcomes.

- A badly priced menu should reduce customer conversion, but should not make the restaurant completely empty.
- A high-price strategy should be viable in Uptown Pre-Theater but risky in Downtown Lunch Rush.
- Events should move demand materially enough that players notice: roughly 15–40% for strong event-dish affinity, not 2–5%.
- A meaningful upgrade should typically be affordable every 60–120 seconds for a healthy restaurant.
- A player should have to choose between immediate operational assistance and upgrades multiple times per match.
- The score difference between similarly skilled players should remain recoverable until the final 20–30% of a match.

## 25. First playable prototype

The very first prototype should be deliberately narrow:

- One market: Downtown Lunch Rush
- One restaurant layout
- Two player restaurants
- Three dishes: Smash Burger, Caesar Salad, Chicken Sandwich
- Two customer segments: Office Worker and Tourist
- One event: Office Break Starts
- One owner action set:
  - Move
  - Cook
  - Deliver
  - Restock
  - Buy upgrade
- One worker type: Cook
- One upgrade: Faster Grill
- One score: Revenue + Satisfaction
- One harness: restaurant layout plus customer flow

## Prototype question

Is it fun to decide whether to personally cook, deliver, restock, or buy an upgrade while competing for the same customers as another player?

If this is not fun, more dishes, markets, assets, and content will not solve the core problem. The prototype should be tested repeatedly with two people as early as possible.

## 26. Example match scenario

### Stadium District: Baseball Game Night

#### Setup

- Market: Stadium District
- Daypart: Evening
- Forecast: High fan traffic, large parties, moderate price sensitivity
- Known likely event: Baseball game ends during the second half of service
- Player menu:
  - Smash Burger — \$15
  - Nachos — \$17
  - Chicken Sandwich — \$14
  - Soda — \$4
- Rival menu:
  - Pasta Primavera — \$21
  - Steak Frites — \$35
  - Caesar Salad — \$13

#### Early service

The player attracts fan groups with burgers and nachos, but the dining room fills quickly. The server begins falling behind on deliveries. The player chooses to deliver ready food rather than cook, preserving freshness and freeing kitchen capacity.

#### Mid-match event

“Baseball Game Ends in 15 seconds.”

The player sees a surge coming. They use accumulated cash to buy a Serving Tray upgrade, then sprint back to the dining area. The rival has high-margin dishes but slow production and begins accumulating frustrated groups.

## Final rush

The player's lower ticket price produces less per order, but they serve significantly more parties with high enough satisfaction. The rival earns more per premium table but loses customers to long projected waits.

## Result

- Player: Higher customers served, higher event objective bonus, moderate margin
- Rival: Higher average ticket, lower throughput, more abandonment
- Player wins composite score through a better match between market, menu, and active execution

## Recommended next step

Build Milestone 0 plus the narrow first playable prototype before producing final art, persistent progression, broad menu content, or elaborate restaurant customization. The highest-value early deliverable is a two-player test where both owners can move around adjacent restaurants, compete for office-lunch customers, and experience one demand surge event.

✱✱

1. <https://discourse.threejs.org/t/cdn-es6-module-import-requires-importmap-vanilla-js/68353>
2. <https://discourse.threejs.org/t/why-does-three-js-require-a-import-map/40308>
3. <https://oneuptime.com/blog/post/2026-02-02-express-websockets/view>
4. <https://karlhadwen.medium.com/node-js-websocket-tutorial-real-time-chat-room-using-multiple-clients-44a8e26a953e>
5. [https://play.google.com/store/apps/details?id=com.dmg.restaurantsimulator&hl=en\\_US](https://play.google.com/store/apps/details?id=com.dmg.restaurantsimulator&hl=en_US)
6. [https://www.reddit.com/r/threejs/comments/11hjuv9/got\\_a\\_very\\_confusing\\_error\\_when\\_converting\\_from/](https://www.reddit.com/r/threejs/comments/11hjuv9/got_a_very_confusing_error_when_converting_from/)
7. <https://learnpurpose.hashnode.dev/game-on-build-a-multiplayer-game-with-nodejs-and-websockets-part-3-of-game-dev-series>
8. <https://steamcommunity.com/app/3058360/discussions/0/599662086417690010/>
9. <https://dev.to/ibmdeveloper/creating-our-websocket-server-creating-a-multiplayer-game-server-part-4-1429>
10. <https://s4565092.stacksofdiscovery.com/stacksofdiscovery-news/restaurant-simulator-games-androids-delicious-delights-1767648243>
11. <https://discourse.threejs.org/t/importing-latest-three-version-with-cdn/43162>
12. <https://www.npmjs.com/package/express-ws>
13. [https://store.steampowered.com/app/1166660/Restaurant\\_Simulator/](https://store.steampowered.com/app/1166660/Restaurant_Simulator/)
14. [https://www.reddit.com/r/gamedesign/comments/292y6c/struggling\\_with\\_making\\_a\\_restaurant\\_simulator\\_fun/](https://www.reddit.com/r/gamedesign/comments/292y6c/struggling_with_making_a_restaurant_simulator_fun/)
15. <https://gist.github.com/asford/5acc2e6ed20f9c2b07d1e0b4a471f0d1>

